ResQCloud

Incident coordination from first report to resolution

ResQCloud is a demonstration platform for coordinating community incident reports. A person submits a report, an authorized operations team reviews and assigns it, a responder records progress, and a supervisor closes the incident. The reporter can follow the report and leave feedback.

This repository is also an AWS integration demonstration. It connects a locally running web app and API to AWS-managed services for the database, event delivery, private evidence, alerts, and optional sign-in. The application is **not hosted on AWS** by this project. Cloud services must be configured in an AWS account before those integrations can be used; local SQLite mode is available for development and for demonstrations without cloud access.

For a teammate: start with the overview and setup below. For console-by-console AWS setup, see the AWS showcase guide. For what is implemented versus still unverified, see implementation status.

What the product does

The product follows a report through an auditable response workflow:

1. A citizen reports an incident, optionally anonymously, and receives a private tracking receipt.
2. A dispatcher reviews the report, records a reason, confirms its details and priority, and assigns a responder.
3. A responder accepts the assignment, updates response progress and records resolution.
4. A supervisor reviews the outcome and closes the incident.
5. The citizen follows status changes and can submit feedback after closure.

The interface has role-based workspaces for citizens, responders, dispatchers, supervisors, and administrators. The API enforces role, ownership, and department access; hiding a link in the interface is not the security control. Reports and transitions are validated, written through database transactions, and recorded in incident history and an event outbox.

This is a faculty/demo platform using fictional data. It is not an emergency dispatch service: the urgent in-app report creates a critical incident inside ResQCloud and does **not** call local emergency services.

AWS connection, in plain language

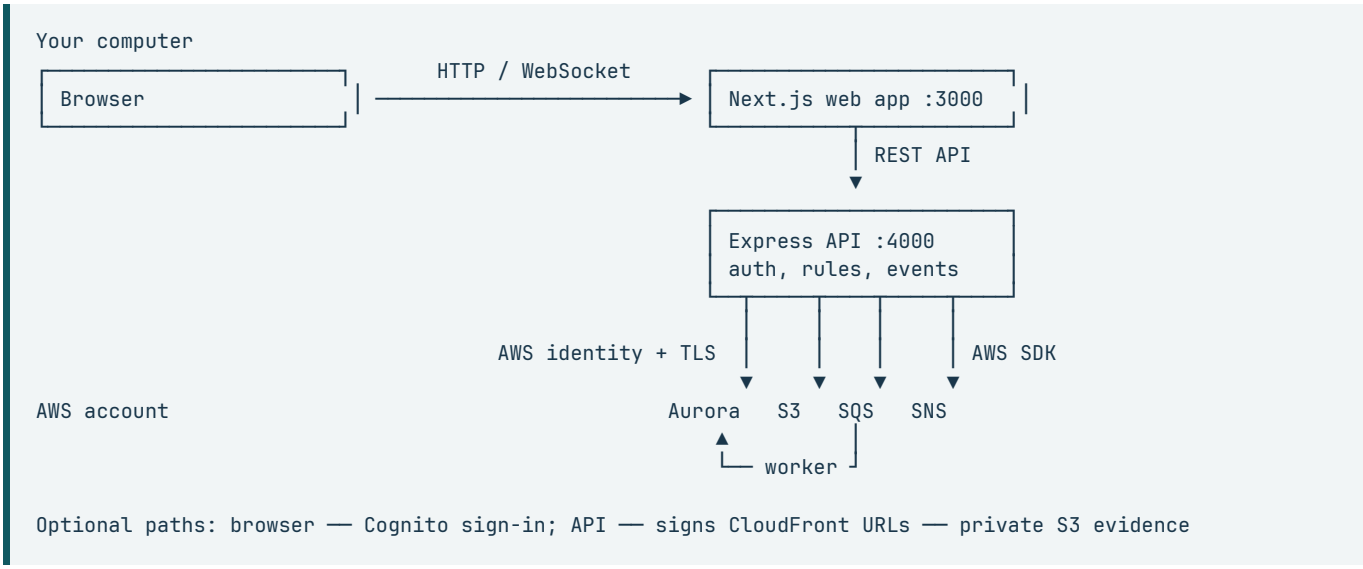
Think of the web app and API as the part your teammate sees and runs. AWS supplies selected managed building blocks behind them:

AWS service	Plain-language role	How ResQCloud uses it
Aurora PostgreSQL	Managed relational database	Stores users, reports, assignments, status history, notifications, and queued events.
Amazon SQS	Durable work queue	Carries incident events from the API to a background consumer so event processing can retry independently.
Amazon S3	Private object storage	Holds incident evidence such as images. The bucket should remain private.
Amazon SNS	Notification publisher	Publishes critical-incident notifications to configured subscribers, such as a confirmed email subscription.
Amazon Cognito *(optional)*	Hosted identity service	Provides citizen sign-in and registration. The API validates Cognito ID tokens before creating an app session.

AWS service	Plain-language role	How ResQCloud uses it
Amazon CloudFront *(optional)*	Content delivery network	Delivers private evidence with short-lived signed URLs while S3 remains private.
Mapbox *(optional, not AWS)*	Map display service	Displays incident locations when a public browser token is configured.

The AWS services are separate resources in an AWS account. ResQCloud does not create them automatically and does not use Terraform. They are configured in the AWS Console using the AWS showcase guide.

What runs where



The local AWS CLI profile supplies temporary AWS credentials to the API's AWS SDK clients. Database access is a separate connection: Sequelize connects to the Aurora PostgreSQL writer endpoint over TLS, and the API can generate a short-lived IAM database authentication token for the configured database user. The token is not a database password stored in `.env`.

Technical event and data flow

1. The browser sends requests to the Next.js app and its API client calls `http://localhost:4000/api/v1`. Protected requests carry a short-lived access token; a same-site, HttpOnly refresh cookie is rotated by the API.
2. Express validates input with shared Zod schemas, authenticates the actor, checks authorization and performs the incident operation.
3. Sequelize stores the change in Aurora PostgreSQL. A domain event is inserted in the same database transaction through the transactional outbox.
4. A single API worker sends outbox events to SQS when configured. Without SQS, local event delivery handles the event directly. The queue consumer validates messages and creates idempotent in-app notifications.
5. Critical events can be published to SNS by the event delivery handler, whether delivery is local or follows SQS consumption. The API sends authorized update notices to a user's WebSocket room; clients then reload data through the authorized REST API.
6. Evidence uploads use a short-lived, size- and content-type-restricted S3 upload policy. Optional CloudFront delivery requires an API authorization check and a five-minute signed URL.

The current worker is designed for one application instance. Queue delivery and SNS publication are at-least-once; notification records guard against duplicate processing where implemented. See architecture, events, and security for implementation details.

Quick start

Requirements

- Node.js 24 or newer and npm.
- Python 3, `make`, and a C++ compiler may be needed if the SQLite native package has no prebuilt binary for your platform.
- No AWS account is required for local development.

Run locally with SQLite

From the repository root:

```
npm ci
node scripts/setupLocal.mjs
npm run db:seed
npm run dev
```

The setup script creates `.env` with a random JWT secret and selects SQLite **only when** `.env` **does not already exist**; it leaves an existing `.env` untouched. If you already have an AWS `.env` and want local SQLite, keep a private backup and switch `DB_DIALECT=sqlite`, `DB_SSL=false`, `DB_SSL_CA_PATH=` and `DB_IAM_AUTH=false` in `.env`. `db:seed` creates the local schema and fictional role accounts. Read `DEV_SEED_PASSWORD` from your private `.env`; do not paste it into source files or commit it.

Open http://localhost:3000. The Next.js web app uses port **3000**; the Express API uses port **4000**. The combined `npm run dev` command starts both. If startup reports that port 3000 is already in use, stop the other app process first; this project's web script is configured for 3000.

Seeded development accounts are:

Role	Email
Citizen	`citizen@resqcloud.test`
Dispatcher	`dispatcher@resqcloud.test`
Responder	`responder@resqcloud.test`
Supervisor	`supervisor@resqcloud.test`
Administrator	`admin@resqcloud.test`

Use the same `DEV_SEED_PASSWORD` for these accounts. The seed is blocked in production.

Try the workflow

1. Sign in as the citizen and submit a fictional incident.
2. Copy the tracking ID and private tracking key from the receipt.
3. Sign in as the dispatcher, review and prioritize the incident, then assign a responder.
4. Sign in as the responder, accept the assignment, record progress, and resolve it.
5. Sign in as the supervisor and close the incident.
6. Return as the citizen to view progress and submit feedback.
7. Review notifications and analytics; use the administrator role for the audit trail.

Connect the app to AWS

The code supports an AWS-backed path using Aurora PostgreSQL and optionally SQS, S3, SNS, Cognito, and CloudFront. AWS resources, permissions, network reachability, and account policy must all be configured first. The AWS showcase guide contains the full console procedure; the key connections are:

1. AWS identity and service permissions

Use the AWS CLI profile for the account that owns or can access the demo resources. For example:

```
aws login --profile resqcloud-demo --region ap-southeast-2
aws sts get-caller-identity --profile resqcloud-demo
```

The API's AWS SDK uses `AWS_PROFILE` to obtain temporary credentials for services such as SQS, S3, and SNS. Do not put AWS access keys in `.env`. Grant only the actions and resource ARNs the demo requires.

Organization accounts can restrict services. An AWS Organizations service control policy (SCP) can explicitly deny an action even when a role appears to have broad access. An account administrator cannot override an SCP deny; an organization administrator must review it, or the demo must use an account where the needed services are allowed.

2. Aurora PostgreSQL

In the repository root `.env`, set `DB_DIALECT=postgres`, the Aurora writer endpoint in `DATABASE_URL`, `DB_SSL=true`, and `DB_SSL_CA_PATH` to a local AWS RDS CA bundle. For the IAM authentication setup described in this repository, set `DB_IAM_AUTH=true`, configure `AWS_REGION`, and grant `rds-db:connect` to the exact database user and cluster resource ID.

The cluster must be **Available** and reachable from the machine running the API. If Aurora is stopped, the API fails during startup because it authenticates to the database before opening port 4000. If it is private, the app must run inside its network or use an approved private connection. A workstation cannot reach a private endpoint just by having AWS credentials.

Apply the schema and development records only after the database and IAM/network access are ready:

```
AWS_PROFILE=resqcloud-demo npm run db:migrate
AWS_PROFILE=resqcloud-demo npm run db:seed
AWS_PROFILE=resqcloud-demo npm run dev
```

3. Optional AWS integrations

- **SQS:** set `SQS_QUEUE_URL`; the API sends outbox events and polls/consumes queue messages. Configure the queue and dead-letter policy in AWS.
- **S3:** set `S3_BUCKET`; the API requests narrowly scoped AWS credentials for private evidence objects. Configure bucket CORS for `http://localhost:3000`.
- **SNS:** set `SNS_TOPIC_ARN` and confirm a subscription if the demonstration should deliver alerts.
- **Cognito:** configure the API-side pool/client identifiers plus the matching public variables `NEXT_PUBLIC_COGNITO_CLIENT_ID` and `NEXT_PUBLIC_COGNITO_DOMAIN`. Register `http://localhost:3000/auth/callback` as a callback URL. The app client is public and uses authorization code with PKCE; never put a Cognito client secret in browser variables.
- **CloudFront:** optional. Set the distribution domain, public key ID, and local private-key path. The S3 bucket stays private; the API signs short-lived viewer URLs after checking access.

`NEXT_PUBLIC_*` values are exposed in the browser bundle and are read when Next.js starts/builds. Restart the dev server after changing them; rebuild a production app. Keep database settings, private signing keys, and AWS credentials out of `NEXT_PUBLIC_*` variables.

Check the connection

```
curl http://localhost:4000/health
curl http://localhost:4000/ready
```

`/health` means the API process is answering. `/ready` also checks that the database can be reached and that the migration table exists. A successful health response alone does not prove Aurora is connected. Run `npm run lint`, `npm run typecheck`, `npm test`, and `npm run build` for local code checks. Browser tests use Playwright and require both local services unless `PLAYWRIGHT_BASE_URL` points at a running web app.

Common startup problems

Symptom	What it means / what to check
Port 3000 already in use	Another web process is running. Stop that process, then restart ResQCloud.
Port 4000 or `/ready` unavailable	The API may have failed its database startup check. Read the API log.
Aurora reports `stopped`	Start the cluster in RDS, wait for **Available**, then restart the local dev command.
`AccessDenied` mentions an SCP	Ask the organization administrator to review the explicit deny. Adding an IAM allow in the member account will not bypass it.
`unable to get local issuer certificate`	Check that `DB_SSL_CA_PATH` points to the correct, current CA bundle. Keep certificate verification enabled.
Connection timeout/reset	Check cluster state, endpoint/region/port, public or private reachability, security group rules, and local network access. AWS credentials do not create a network path into a VPC.
`NEXT_PUBLIC_*` config appears stale	Restart Next.js (or rebuild) after changing the value.

Project structure

Path	Responsibility
`apps/web/app/`	Next.js routes, page shells, and global styles.
`apps/web/features/`	Incident reporting, tracking, dashboards, admin, auth, and role views.
`apps/web/components/`	Shared UI and application-wide auth provider.
`apps/web/lib/`	Browser API client, Cognito flow, offline queue, and utilities.
`apps/api/src/`	Express routes, auth, authorization, database models/migrations, AWS adapters, and event worker.
`packages/shared/`	Shared TypeScript types, Zod schemas, and workflow rules.
`tests/`	API/domain/unit tests and Playwright browser tests.
`docs/`	AWS setup, architecture, security, status, and operational notes.

Useful commands:

```
npm run dev          # web :3000 + API :4000
npm run db:seed      # migrate and create fictional demo users/incidents
npm run db:migrate   # apply database migrations
npm run lint         # ESLint and Prettier checks
npm run typecheck    # TypeScript checks
npm test             # unit/domain/API tests
npm run build        # production API and web builds
npm run test:e2e     # Playwright browser tests
```

Project boundaries and demo safety

This is a functional local demo with AWS integration code; it is **not a production-ready emergency response system**. The integrations require account resources and live verification. Local tests use SQLite, not Aurora. Features such as S3 upload, CloudFront, Cognito, Maps, SQS and SNS do not become operational simply because their packages are installed; each needs valid service settings and permissions.

The current deployment guide does not configure AWS compute, load balancing, production secrets, horizontal worker coordination, or disaster recovery. Other documented production gates include MFA enrollment/recovery, evidence malware scanning, scale-out coordination, performance/security review, and operational recovery testing. Check implementation status before describing capabilities as complete.

Use fictional sample data only. Do not put real emergency, medical, or identifying information into this demo. Check current AWS pricing and account eligibility before creating resources; a budget alert notifies you but does not cap charges. After the demonstration, stop or delete resources as described in the AWS showcase cleanup steps.

Further reading

- Console-managed AWS setup
- Aurora PostgreSQL and IAM database authentication
- AWS architecture
- Application architecture
- API reference
- Security and privacy
- Implementation status and release gates
- Deployment boundary